

Kisan Dost

An AI agronomy agent that helps Pakistani farmers decide what to grow, how to protect it, and when to sell — built with the OpenAI Agents SDK.

Build type Terminal-based agent	Framework OpenAI Agents SDK (Python)	Backend / Frontend Optional (bonus marks)	Grading On the spot, live
---	---	---	-------------------------------------

Same problem for everyone. Your creativity decides how far you take it.

THE PROBLEM WORTH SOLVING

Why Kisan Dost matters

Agriculture feeds Pakistan and employs a huge share of its people, yet most smallholder farmers make season-defining decisions — what to plant, how much fertilizer to buy, which pest is eating the crop, when and where to sell — with almost no expert on hand. One government extension officer may be responsible for thousands of farmers. That gap is real, expensive, and solvable with software.

Your job is to build **Kisan Dost** (“Farmer’s Friend”): a terminal advisory line where a farmer or a field officer types a question in plain Urdu-English and gets a genuinely useful answer back. Think of it as a 24/7 agronomy helpline powered by an AI agent.

The demo moment you are aiming for

A farmer in Multan asks what to plant this Rabi season on 5 acres with limited water. Kisan Dost recommends crops with profit estimates, returns a fertilizer plan in bags of Urea and DAP with total cost, flags a whitefly risk with a safe treatment, and points to the fertilizer subsidy to apply for — all from one typed question. That is a moment you can show anyone with pride.

Why this is a strong Agentic AI problem

- It breaks down naturally into many **tools** and even several **specialist agents**, so it stretches every part of the Agents SDK.
- It has a built-in difficulty gradient: a weaker student still ships a working single agent; a strong student builds a guarded multi-agent system. That makes live grading fair.
- It is unmistakably Pakistani and useful — not another to-do bot — so students present it with real confidence.

YOUR MISSION

What the agent must do

Each capability below should be a clean, separate **function tool**. You do not need a real internet connection for any of them — realistic hardcoded tables or small CSV files are perfectly acceptable. Real APIs and datasets (listed at the end) earn bonus marks.

#	Tool	What it does
1	Crop Advisor	Given district, soil type, season (Rabi / Kharif), water availability and land size, recommend the best crops with expected yield and profit.
2	Pest & Disease Doctor	Farmer describes symptoms (“cotton leaves curling, tiny white insects”); identify the likely pest or disease and give treatment plus a SAFE dosage.
3	Fertilizer Calculator	Compute NPK need per acre for a crop and convert it into bags of Urea / DAP with the total cost in rupees.
4	Mandi Price Lookup	Return current or typical wholesale prices in nearby mandis so the farmer knows when and where to sell.
5	Irrigation & Weather	Given the forecast and crop stage, advise irrigation timing and warn about frost or heatwave risk.
6	Profit Estimator	Full season budget: input costs vs expected revenue, giving net margin and break-even yield.

#	Tool	What it does
7	Govt Support Finder	Surface the Kisan Card, subsidized fertilizer or agri-loan schemes relevant to the farmer's province and need.

Build at least four of these well. More tools help only if they are genuinely useful and clean — quality beats quantity.

TECHNICAL EXPECTATIONS

How this maps to the OpenAI Agents SDK

This is exactly what I will be inspecting in your code. Treat it as a checklist.

Function tools

Every capability as a typed `@function_tool`. I will look at input validation, clear docstrings, and single responsibility per tool.

Structured outputs

Use Pydantic `output_type` models (e.g. a `CropPlan` or `FertilizerPlan`) instead of loose text. This separates serious builds immediately.

Multi-agent handoffs

A triage agent that routes to specialists — Agronomy Agent, Pest Doctor, Market Agent, Finance Agent. This is the single biggest complexity differentiator.

Guardrails

An input guardrail that rejects off-topic or unsafe requests, and an output guardrail that keeps pesticide dosages within safe limits and refuses human-medical advice. Safety-aware agents score high.

Sessions and context

Use Sessions so the farmer's district, land size and crop are remembered across turns, and carry a typed farmer-profile object as context. No re-typing the same details every message.

Tracing (bonus)

Show that you inspected your agent runs with the SDK's built-in tracing.

Minimal shape to get started (structure only — the thinking is still yours):

```
pip install openai-agents

from agents import Agent, Runner, function_tool
from pydantic import BaseModel

class FertilizerPlan(BaseModel):
    urea_bags: float
    dap_bags: float
    total_cost_pkr: int

@function_tool
def fertilizer_calculator(crop: str, acres: float) -> FertilizerPlan:
    """Return bags of Urea/DAP and total cost for a crop and land size."""
    ... # your logic / lookup table here

agronomy_agent = Agent(name='Agronomy Agent', tools=[fertilizer_calculator])
# then a triage Agent(...) with handoffs=[agronomy_agent, pest_agent, ...]
```

HOW YOU WILL BE GRADED

Live, on-the-spot marking

I will read your code with you and mark on quality, working functions, structure and safety. The tiers below tell you where you stand before you even present.

Tier	What it looks like
Pass	A single agent, 3–4 working tools, a clean terminal loop that runs without crashing.
Good	Structured Pydantic outputs, session memory, 6+ useful tools, solid input validation and error handling.
Excellent	Multi-agent handoffs, working guardrails (especially pesticide safety), the profit model, plus any real API, Urdu support, or a backend/frontend.

Detailed rubric (100 points)

Area	What I check	Pts
Code quality	Readable, well-named, no dead code, sensible files, docstrings, error handling.	20
Working functions	Tools actually run and return correct, useful results on live inputs.	20
Agent design	Sensible tools/handoffs, structured outputs, good instructions and routing.	20
Safety & guardrails	Rejects off-topic/unsafe input; safe pesticide limits; no medical advice.	15
Problem impact	Does it truly help a real Pakistani farmer? Realism of data and advice.	15
Creativity / bonus	Urdu output, real APIs, memory, backend/frontend, tracing, polish.	10

MAKE IT YOURS

Same problem, your own build

Everyone solves Kisan Dost, but no two solutions should look alike. Some directions you can own:

- Specialise by crop and region — cotton in Sindh, wheat or maize in Punjab.
- Add Urdu voice or Urdu text output for farmers who prefer it.
- Build a WhatsApp-style conversational flow in the terminal.
- Go deep on the financial modelling and break-even analysis.
- Design for low connectivity: offline-first data for remote areas.
- Wire in a real weather or mandi-price API instead of mock data.

Rules & deliverables

- Core build must be a terminal agent** using the OpenAI Agents SDK. Backend and frontend are optional and count as bonus, not requirements.
- Write your own code. Using the docs and reading examples is encouraged; copying a finished project is not.
- Keep API keys in a `.env` file — never hard-code or commit them.

Submit a public GitHub repo with a clear README: how to run it, what each tool does, and which APIs or datasets you used.

Be ready to run it live and walk me through the code — that is where the marks are given.

No OpenAI credits? You can still compete.

The Agents SDK is provider-agnostic and works with any OpenAI-compatible endpoint. You can point it at free or low-cost providers (for example a Gemini or Groq OpenAI-compatible base URL) by setting the base URL and key. Focus your marks on tool and agent design — the model behind it can be swapped.

REAL TOOLS TO BUILD WITH

APIs, sites & datasets (all verified)

Every link below is real and working. Mock data is fine to pass; wiring any of these in earns bonus marks.

Core framework

OpenAI Agents SDK — official docs

<https://openai.github.io/openai-agents-python/>

Agents, function tools, handoffs, guardrails, sessions and tracing. Install with `pip install openai-agents`. Your primary reference.

OpenAI API keys

<https://platform.openai.com/api-keys>

Create the key your agent will use. Store it in a `.env` file.

Weather & irrigation

Open-Meteo Weather API (free, no key)

<https://open-meteo.com/>

Free forecast and historical weather, no sign-up. Example for Faisalabad: `api.open-meteo.com/v1/forecast?latitude=31.45&longitude=73.13¤t=temperature_2m,relative_humidity_2m&daily=precipitation_sum,temperature_2m_max`

Open-Meteo Geocoding API

<https://geocoding-api.open-meteo.com/v1/search?name=Multan>

Turn a Pakistani city or district name into latitude/longitude for the weather call.

Market / mandi prices

AMIS Punjab — daily mandi prices

<http://www.amis.pk/>

Official daily wholesale rates for fruits, vegetables and commodities across 100+ Punjab markets, including Faisalabad. Great source to scrape or model your price tool on.

Agriculture Department, Punjab

<https://www.agripunjab.gov.pk/>

Provincial schemes, crop guidance and the Kisan Card programme for your Govt Support tool.

Datasets (for crop & disease logic)

Crop Recommendation Dataset (Kaggle)

<https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>

2,200 rows mapping soil N-P-K, temperature, humidity, pH and rainfall to 22 crops (rice, maize, cotton, chickpea, lentil and more). Perfect for the Crop Advisor tool.

PlantVillage Dataset (crop leaf diseases)

<https://github.com/spMohanty/PlantVillage-Dataset>

Tens of thousands of labelled crop-leaf images across many diseases — a strong base for the Pest & Disease Doctor (also mirrored on Kaggle).

FAOSTAT — Pakistan crop statistics

<https://www.fao.org/faostat/en/#country/165>

Official production, yield and area data for Pakistan to ground your yield and profit estimates in real numbers.

Pakistan Bureau of Statistics — Agriculture

<https://www.pbs.gov.pk/>

National agricultural statistics and prices for realistic assumptions.

Build something a farmer would actually thank you for. Good luck.